

Security and Privacy in Federated Learning

Jonas Müller, Rebekka Schnoor

21.05.2026

Agenda

- Secure Aggregation
 - Motivation
 - Threat Models
 - Protocol Examples

- Robust Aggregation
 - Motivation
 - Threats
 - Robust Aggregation Rules
 - Trust and Verification

- Privacy-Robustness-Utility Tradeoff

Secure Aggregation

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Threat Models

- Honest-but-curious

The participant follows the protocol honestly but tries to learn as much as possible from the received messages

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Threat Models

- **Honest-but-curious**

The participant follows the protocol honestly but tries to learn as much as possible from the received messages

- **Malicious**

The participant can lie or break the protocol

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Threat Models

- **Honest-but-curious**

The participant follows the protocol honestly but tries to learn as much as possible from the received messages

- **Malicious**

The participant can lie or break the protocol

- **Malicious + (limited) access to memory**

The malicious participant additionally has access to some of the other participant's local memory (i.e. learns their secrets)

Secure Aggregation

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Threat Models

- **Honest-but-curious**
Follow protocol but maximize learning
- **Malicious**
Can break the protocol
- **Malicious + (limited) access to memory**
+ Partial access to secrets

Challenges

- **User dropout**
- **Limited bandwidth and/or computation power**
- **Transmission of high dimensional information vectors (gradients)**
- **Unauthenticated network model**

Secure Aggregation Protocol by Bonawitz et al. [3]

Protocol Summary

- Uses masking vectors to encrypt individual gradients that cancel out in the aggregation
- Robust to user dropout by using secret sharing and a second masking vector
- Secure against malicious adversaries with limited memory access by using OneTime Pads and a sufficiently high threshold for secret reconstruction

Costs

for n users and vector dimension k

computation

User $O(n^2 + kn)$

Server $O(kn^2)$

communication

User $O(n + k)$

Server $O(n^2 + kn)$

storage

User $O(n + k)$

Server $O(n^2 + k)$

2 Main Variants:

■ Fully Homomorphic Encryption (FHE)

Uses additively homomorphic encryption scheme

1. Clients send encrypted gradients to the server
2. Server aggregates encrypted gradients and shares result
3. Clients jointly decrypt the updated model

2 Main Variants:

■ Fully Homomorphic Encryption (FHE)

Uses additively homomorphic encryption scheme

1. Clients send encrypted gradients to the server
2. Server aggregates encrypted gradients and shares result
3. Clients jointly decrypt the updated model

■ Secure Two / Multi-Party Computation (2PC/MPC)

1. Client(s) split updates among servers using secret sharing
2. Servers jointly aggregate and distribute updates
3. Client(s) reconstruct the result

2 Main Variants:

■ Fully Homomorphic Encryption (FHE)

Uses additively homomorphic encryption scheme

1. Clients send encrypted gradients to the server
2. Server aggregates encrypted gradients and shares result
3. Clients jointly decrypt the updated model

■ Secure Two / Multi-Party Computation (2PC/MPC)

1. Client(s) split updates among servers using secret sharing
2. Servers jointly aggregate and distribute updates
3. Client(s) reconstruct the result

+ Privacy preservation against malicious servers

! Assumes honest clients

High communication costs

MPC/2PC variant assumes non-colluding servers

Motivation

The aggregation protocols should be robust against malicious clients trying to manipulate the training process.

Main Threats

- **Poisoning Attacks**
 - **Backdoor:** Implement sensitivity to a specific trigger
 - **Untargeted:** Degrade overall model performance
 - **Targeted:** Degrade performance on specific class(es)

Defense Mechanisms

- **Robust aggregation rules**
 - Krum
 - Trimmed Mean
 - FLTrust
- **Anomaly detection on client updates**

Robust Aggregation Rules

- **Krum [2]:** Only consider updates that are closest to other updates
 - ! Assumes few malicious clients and that honest updates cluster together → fails for non-IID data
- **Trimmed Mean [7]:** For each coordinate of the update vector, cut the top and bottom $n\%$ before averaging
 - ! neglects correlations between dimensions
for fixed, known thresholds, malicious updates can be crafted accordingly
- **FLTrust [5]:** Maintain small root dataset on server to calculate update anchor, weigh client updates by similarity to anchor
 - ! Assumes trusted, representative dataset available

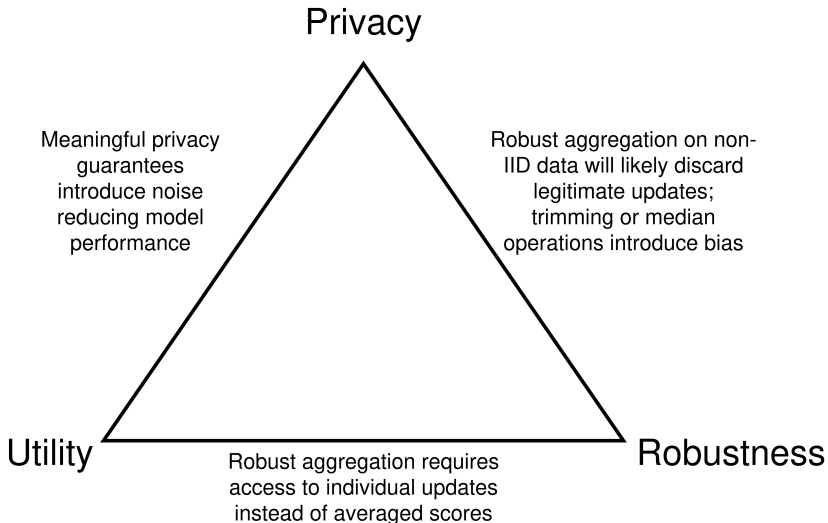
Robust Aggregation Rules

- **Krum [2]:** Only consider updates that are closest to other updates
 - ! Assumes few malicious clients and that honest updates cluster together → fails for non-IID data
- **Trimmed Mean [7]:** For each coordinate of the update vector, cut the top and bottom $n\%$ before averaging
 - ! neglects correlations between dimensions
for fixed, known thresholds, malicious updates can be crafted accordingly
- **FLTrust [5]:** Maintain small root dataset on server to calculate update anchor, weigh client updates by similarity to anchor
 - ! Assumes trusted, representative dataset available

Under highly non-IID data, honest clients naturally produce very different updates, making it hard to distinguish them from dishonest ones

- **Zero-Knowledge-Proofs** to prove that updates satisfy certain requirements [4]
 - ! Computationally expensive for large updates
- **Verifiable Computation:** Client proves it ran the training algorithm correctly, not just that the output has certain properties
 - ! Proof generation cost grows exponentially with number of gradient steps
- **Auditing/Attestation Frameworks:** audit logs + secure hardware attestation instead of cryptographic proofs
 - ! Requires legal accountability

Privacy-Robustness-Utility Tradeoff [1]



References

- [1] Youssef Allouah u. a. “On the Privacy-Robustness-Utility Trilemma in Distributed Learning”. In: *Proceedings of the 40th International Conference on Machine Learning*. Proceedings of Machine Learning Research. 2023. URL: <https://arxiv.org/abs/2302.04787>.
- [2] Peva Blanchard, Rachid Guerraoui und Julien Stainer. “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent”. In: *Advances in Neural Information Processing Systems*. Bd. 30. 2017, S. 119–129.
- [3] Keith Bonawitz u. a. *Practical Secure Aggregation for Federated Learning on User-Held Data*. 2016. arXiv: 1611.04482 [cs.CR]. URL: <https://arxiv.org/abs/1611.04482>.

References

- [4] Lukas Burkhalter u. a. “RoFL: Attestable Robustness for Secure Federated Learning”. In: *IEEE Symposium on Security and Privacy (SP)*. 2023. URL: <https://arxiv.org/abs/2107.03311>.
- [5] Xiaoyu Cao u. a. “FLTrust: Byzantine-Robust Federated Learning via Trust Bootstrapping”. In: *Network and Distributed System Security Symposium (NDSS)*. 2021. URL: <https://arxiv.org/abs/2012.13995>.
- [6] Hossein Fereidooni u. a. *SAFELearn: Secure Aggregation for private FEderated Learning*. Cryptology ePrint Archive, Paper 2021/386. 2021. URL: <https://eprint.iacr.org/2021/386>.
- [7] Dong Yin u. a. “Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates”. In: *Proceedings of the 35th International Conference on Machine Learning*. Bd. 80. Proceedings of Machine Learning Research. PMLR, 2018, S. 5650–5659.

Secure Aggregation Protocol by Bonawitz et al. [3]

1. Users pairwise agree on input perturbation pair $p_{u,v}, p_{v,u}$
2. Each user masks their gradient using the sum of all perturbations known and sends the result to the Server
3. The Server aggregates as usual because the perturbations are structured to cancel each other

Secure Aggregation Protocol by Bonawitz et al. [3]

1. Users pairwise agree on input perturbation pair $p_{u,v}, p_{v,u}$
 2. Each user masks their gradient using the sum of all perturbations known and sends the result to the Server
 3. The Server aggregates as usual because the perturbations are structured to cancel each other
- + information theoretically secure through OneTime Pads, even for malicious adversaries with limited memory access

Secure Aggregation Protocol by Bonawitz et al. [3]

1. Users pairwise agree on input perturbation pair $p_{u,v}, p_{v,u}$
 2. Each user masks their gradient using the sum of all perturbations known and sends the result to the Server
 3. The Server aggregates as usual because the perturbations are structured to cancel each other
- + information theoretically secure through OneTime Pads, even for malicious adversaries with limited memory access
 - ! strong assumptions:
 - pairwise secure channels with sufficient bandwidth for secret exchange
 - protocol completion by all users

Secure Aggregation Protocol by Bonawitz et al. [3]

0. Each user creates a key pair and broadcasts their public key to all other users
1. Perturbation vector agreement
 - 1.1 Secret sharing: Users distribute perturbation vector among all other users with reconstruction threshold t
2. Masking + sending of gradients
 - 2.1 Only use perturbations shared between surviving users. Reveal secrets of dropped out users if at least t users survived
3. Aggregation by server

Secure Aggregation Protocol by Bonawitz et al. [3]

- 0. Each user creates a key pair and broadcasts their public key to all other users + Robustness to user dropout
- 1. Perturbation vector agreement
 - 1.1 Secret sharing: Users distribute perturbation vector among all other users with reconstruction threshold t
- 2. Masking + sending of gradients
 - 2.1 Only use perturbations shared between surviving users. Reveal secrets of dropped out users if at least t users survived
- 3. Aggregation by server

Secure Aggregation Protocol by Bonawitz et al. [3]

- 0. Each user creates a key pair and broadcasts their public key to all other users
 - 1. Perturbation vector agreement
 - 1.1 Secret sharing: Users distribute perturbation vector among all other users with reconstruction threshold t
 - 2. Masking + sending of gradients
 - 2.1 Only use perturbations shared between surviving users. Reveal secrets of dropped out users if at least t users survived
 - 3. Aggregation by server
- + Robustness to user dropout
 - ! Secret exposure in case of malicious server or delayed update sending; increased communication cost

Secure Aggregation Protocol by Bonawitz et al. [3]

0. Key pair generation + generation of masking vector b
1. Perturbation vector agreement
 - 1.1 Secret sharing including b
2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
3. Aggregation by server

Secure Aggregation Protocol by Bonawitz et al. [3]

0. Key pair generation + generation of masking vector b
 1. Perturbation vector agreement
 - 1.1 Secret sharing including b
 2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
 3. Aggregation by server
- + For $t > \frac{n}{2}$, all gradients stay masked by either p or b ; extend to stronger adversaries by increasing threshold

Secure Aggregation Protocol by Bonawitz et al. [3]

0. Key pair generation + generation of masking vector b
 1. Perturbation vector agreement
 - 1.1 Secret sharing including b
 2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
 3. Aggregation by server
- + For $t > \frac{n}{2}$, all gradients stay masked by either p or b ; extend to stronger adversaries by increasing threshold
 - ! Communication cost of $O(kn^2)$

Secure Aggregation Protocol by Bonawitz et al. [3]

0. Key pair generation + generation of masking vector b
 1. Perturbation vector agreement
 - 1.1 Secret sharing including b
 2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
 3. Aggregation by server
- + For $t > \frac{n}{2}$, all gradients stay masked by either p or b ; extend to stronger adversaries by increasing threshold
 - ! Communication cost of $O(kn^2)$

Reducing Complexity

Share seeds for PRG instead of full vectors using DH Key Exchange via server

Secure Aggregation Protocol by Bonawitz et al. [3]

0. Key pair generation + generation of masking vector b
 1. Perturbation vector agreement
 - 1.1 Secret sharing including b
 2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
 3. Aggregation by server
- + For $t > \frac{n}{2}$, all gradients stay masked by either p or b ; extend to stronger adversaries by increasing threshold
- ! Communication cost of $O(kn^2)$

Reducing Complexity

Share seeds for PRG instead of full vectors using DH Key Exchange via server

Getting rid of secured channel assumption

Setup secured channels via server-mediated DH $\rightarrow$ adds PFS